

- Algorithmique et Structures de Données (SD)-
MIP-MATHEMATIQUES (Semestre 4)

TD/TP n° 2: Listes chaînées : Implémentation et opérations
(Éléments de corrigé)

```
#include <stdio.h>
#include <stdlib.h>
```

//Exercice 1 - 1

```
struct maillon {
    int info;
    struct maillon *suiv;  };
```

```
typedef struct maillon* liste;
```

//Exercice 1 - 2

```
liste tete(){ return NULL;}
```

//Exercice 1 - 3

```
liste ajouter_en_tete(int h ,liste L){
```

//L'élément à insérer sera la nouvelle tête.

```
    liste tete=(liste)malloc(sizeof( struct maillon));
    tete->info= h;
    tete->suiv = L;

    return tete;
}
```

//Exercice 1 - 4

```
liste free_liste(liste L){
```

```
    liste q=L;
```

```
    while(L!=NULL){
        q=L->suiv;
        free(L);
        L=q;
    }
```

```
    return NULL;
```

```
}
```

//Exercice 1 - 5

```
liste copie_liste(liste L)
{
    liste L2,n;
    if(L==NULL)
        L2=NULL;
    else{
        L2=malloc(sizeof(struct maillon));
        n=L2;
        while(L!=NULL){
            *n=*L;
            L=L->suiv;
            if (L!=NULL){
                n->suiv=malloc(sizeof(struct maillon));
                n=n->suiv;
            }
        }
    }
    return L2;
}
```

//Exercice 2 - 1

```
int longueur(liste L){
    int n=0;

    while(L!=NULL){
        n++;
        L = L->suiv;
    }
    return n;
}
```

// Exercice 2 - 2

```
int membre(liste L, int valeur){

    while(L!=NULL){

        if(L->info==valeur) {printf("\n\n l'element %d EST membre de la liste.\n\n ", valeur);
        return 1;}
        L = L->suiv;
    }
    printf("\n\n l'element %d N EST PAS membre de la liste.\n\n ", valeur);
    return 0;
}
```

```
int occurrence(liste L, int valeur){
    int compteur=0;
    while(L!=NULL){

        if(L->info==valeur) compteur++;
        L = L->suiv;
    }

    return compteur;
}
```

// Exercice 2 - 3

```
void afficher_liste(liste L){
    printf("\n\n --AFFICHAGE LISTE--\n\n");
    if (L==NULL)
        printf("\n LA LISTE EST VIDE \n");

    else{
        liste p=L;
        // printf("|");
        while(p!=NULL){
            printf("|%d|->", p->info);
            p = p->suiv;
        }
        printf("NULL\n");
    }
}
```

// Exercice 2 - 4

```
int egalite(liste L1,liste L2){
    int i;
    if (longueur(L1)!=longueur(L2)) {printf("\n Les deux listes NE SONT PAS egaux \n");
return -1;}
    else
        for( i=0; i<longueur(L1);i++)
            if(L1->info!=L2->info) {printf("\n Les deux listes NE SONT PAS egaux \n"); return -
1;}

    printf("\n Les deux listes SONT egaux \n");
    return 1;
}
```

// Exercice 2 - 5

```
int trouver_rang(liste L,int rang){
    int i;
    if(rang>longueur(L)) printf("\n Rang INVALIDE \n");
    else
    for( i=1; i<rang;i++)
        L=L->suiv;

    printf("\n L'elemnt de rang %d est %d \n", rang, L->info);
    return L->info;
}
```

// Exercice 3 - 1

```
liste supprimer_premier (liste L){
    liste courant;

    if (longueur(L)){
        courant=L;// Action 1
        L=L->suiv; // Action 2
        free(courant); // Action 3
    }
    return L;
}
```

// Exercice 3 - 2

```
liste supprimer (liste L, int k ){
    liste prec, courant;
    int i;
    if (k>=0 && k <longueur(L)){
        if (k==0){//suppression du premier maillon
            courant=L;// Action 1
            L=courant->suiv; // Action 2
            free(courant); // Action 3
        }else{// Suppression des autres maillons
            i=0; courant=L;
            while (i<k){//Actions 1 et 2
                prec = courant;
                courant=courant->suiv;
                i=i+1;
            }
            prec->suiv = courant->suiv;// Action 3
            free(courant);// Action 4
        }
    }
    return L;
}
```

// Exercice 3 - 3

```
liste inserer (liste L, int k ,int e){
    liste nouv, courant;
    int i;
    if (k>=0 && k <=longueur(L)){
        nouv=(liste)malloc(sizeof(liste)); //Action 1
        nouv->info=e; // Action 2
        if (k==0){ //insertion au début
            nouv->suiv=L;
            L=nouv;
        }else{ // insertion
            i=0; courant=L;
            while (i<k-1){
                courant=courant->suiv;
                i=i+1;
            }
            nouv->suiv = courant->suiv; // Action 3
            courant->suiv = nouv; // Action 4
        }
    }
    return L;
}
```

// Exercice 4

```
liste addition(liste L1, liste L2){
    liste L3=tete();
    int i;
    int l=longueur(L1);
    if(l!=0){
        for(i=0;i<l;i++){
            L3=inserer(L3,longueur(L3),L1->info+L2->info);
            L1=L1->suiv;L2=L2->suiv;
        }
    }

    return L3;
}
```

PROGRAMMES DE TEST POUR CHAQUE EXERCICE

// Exercice 1

```
int main(int argc, char *argv[]){
{ int a=1;
  liste L;
  L=tete();

  L=ajouter_en_tete(1 , L);
  L=ajouter_en_tete(2 , L);

  liste L2=copie_liste(L);
  L2=free_liste(L2);

  system("PAUSE");
  return 0;
}
```

// Exercice 2

```
int main(int argc, char *argv[]){

  printf("LONGUEUR DE L = %d \n", longueur(L));
  membre(L2, 12);
  afficher_liste(L);
  afficher_liste(L2);
  printf ("\n\n Le nombre d'occurrence de %d est %d \n\n ",a,occurrence(L, a));

  liste L3;   L3=tete();

  L3=ajouter_en_tete(1 , L3);
  L3=ajouter_en_tete(2 , L3);

  liste L4=copie_liste(L3);

  egalite(L3,L4);

  L4=ajouter_en_tete(3 , L4);
  L4=ajouter_en_tete(4 , L4);

  afficher_liste(L4);
  trouver_rang(L4, 2);

  system("PAUSE");
  return 0; }
```

// Exercice 3

```
int main(int argc, char *argv[]){
    liste L5;
    L5=tete();
    L5=ajouter_en_tete(10 , L5);
    L5=ajouter_en_tete(20 , L5);
    L5=ajouter_en_tete(30 , L5);
    L5=ajouter_en_tete(40 , L5);
    L5=ajouter_en_tete(50 , L5);
    afficher_liste(L5);
    L5=supprimer_premier (L5);
    afficher_liste(L5);
    L5=supprimer(L5,2);
    afficher_liste(L5);
    L5=insérer(L5,2,20);
    L5=insérer(L5,0,50);
    afficher_liste(L5);
    L5=insérer(L5,longueur(L5),-10);
    afficher_liste(L5);

    system("PAUSE");
    return 0;
}
```

// Exercice 4

```
int main(int argc, char *argv[]){
    liste L6,L7,L8;
    L6=tete(); L7=tete(); //L8=tete();
    L6=ajouter_en_tete(10 , L6);
    L6=ajouter_en_tete(20 , L6);
    L6=ajouter_en_tete(30 , L6);
    L7=ajouter_en_tete(50 , L7);
    L7=ajouter_en_tete(60 , L7);
    L7=ajouter_en_tete(70 , L7);

    L8=addition(L6, L7);
    afficher_liste(L6);
    afficher_liste(L7);
    afficher_liste(L8);

    system("PAUSE");
    return 0;
}
```